PROJET — Mettre en production le portail de suivi des tournées

> **Module** : Administration d'un Linux Server · BTC2 **Jours** : 4/5 et 5/5 — **deux jours**, du **jeudi** (lancement à 10 h 30) au **vendredi 13 h 00** (rendu) · **Travail individuel Rendu** : dans votre dépôt Git, dossier `PROJET/` (dernier commit poussé **avant vendredi 13 h 00**) — **pas de soutenance**, **pas de grille de notation** dans cette fiche **Rattachement syllabus** : Mode d'évaluation « Projet » · synthèse des Programmes I à IV. Ce projet remplace le TP4 du planning. **Environnement** : `srv-core` (cible), `srv-edge` pour un test pare-feu — voir README §3

> ⚠️ **Ceci est une fiche projet, pas un TP.** Aucune commande n'est donnée : vous avez tout pratiqué dans les TP1 à TP3 (et dans le guide du TP4 pour la supervision). Le projet ne demande **aucune compétence nouvelle**, à une exception près : l'installation de l'agent Zabbix (E8), pour laquelle le guide du TP4 vous est fourni. Il demande d'assembler ce que vous savez faire sur un service neuf, de le rendre exploitable par quelqu'un d'autre, et de **prouver** que chaque exigence est tenue. Chaque exigence indique le TP où vous avez appris à la satisfaire. Le TP4 n'est plus au planning : son énoncé reste disponible comme **guide de référence** pour E8 (agent Zabbix, `sar`).

Contexte

Les exploitants d'Hélios Logistique veulent savoir, depuis le réseau métier, si le collecteur de télémétrie des camions (TP1) est vivant, sans ouvrir de session SSH sur `srv-core`. La DSI a choisi la solution la plus simple possible : un **portail de suivi**, c'est-à-dire un petit serveur HTTP qui publie une page d'accueil et un fichier `etat.json` régénéré toutes les deux minutes à partir de la télémétrie.

Le code n'est pas le sujet : le serveur HTTP est celui de Python, livré avec Rocky Linux. Le sujet, c'est **la mise en production**. Le portail doit redémarrer seul, ne voir que ce qu'il doit voir, n'être joignable que du réseau métier et être supervisé. Il doit aussi se restaurer après un incident, et sa conformité doit se vérifier en une commande. Léa Fabre, nouvelle responsable du suivi des tournées, en sera l'exploitante. Elle n'est pas administratrice.

Le directeur technique a prévenu : **« Je ne lirai pas votre configuration. Je lancerai votre script d'audit, je casserai deux choses, et je le relancerai. »** *(Le script d'audit est le bonus E10 : sans lui, le formateur lance son propre contrôle.)*

Objectifs

- Livrer un service systemd durci, supervisé et sauvegardé, sur un serveur déjà sécurisé, **sans régression** sur ce qui existait (TP1 à TP3).
- Concevoir un modèle de droits minimal (comptes, setgid, ACL, sudo) pour un service et son exploitante.
- Écrire deux scripts de production (publication, sauvegarde). *(Bonus : un script d'audit de conformité.)*

- Prouver une restauration. *(Bonus : rédiger le runbook d'astreinte du service.)*

Prérequis

`srv-core` et `srv-edge` dans l'état de fin du TP3 (instantané `03-fin-TP3`). Si votre `srv-core` est dans un autre état, restaurez cet instantané. **Prenez ensuite l'instantané `04-debut-projet`.**

Contrôlez ces points avant de commencer. Tout écart doit être corrigé (ou signalé au formateur) avant la fin de la première heure de travail :

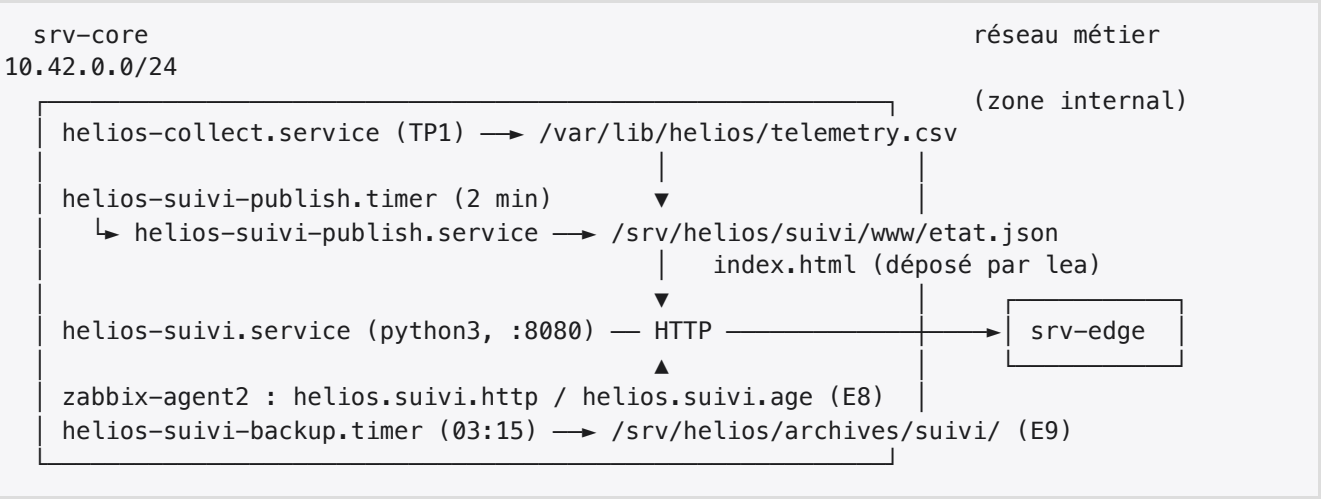
Contrôle	Attendu	Vient du
`systemctl is-active helios-collect.service`	`active`, et `/var/lib/helios/telemetry.csv` grossit	TP1
`systemctl cat helios.target`	la cible métier existe	TP1
`firewall-cmd --get-default-zone` · `--get-active-zones`	`drop` · `bond0` et `bond0.42` dans `internal`	TP3
`sshd -T | grep -E '^(port|allowgroups|passwordauthentication) '`	`2242` · `ssh-admins` · `no`	TP3
`systemctl is-active helios-watch-auth.timer`	`active`	TP3

Ne sont **pas** fournis au départ, vous les mettez en place dans le projet : l'agent Zabbix (`zabbix-agent2`), `zabbix-get`, la collecte `sysstat` et le répertoire de sauvegarde `/srv/helios/archives`.

> ⚠ **Installez l'agent Zabbix en premier** (E8, point 1), avant E2 : c'est l'installation du paquet qui crée le compte `zabbix`, et l'ACL de E2 (point 3) ne peut pas être posée tant qu'il n'existe pas.

---

1. Architecture cible



2. Paramètres imposés

Le script de contrôle du formateur s'appuie sur ces noms : **ne les changez pas**.

Élément	Valeur
Groupe de l'équipe suivi	`suivi`, gid `5003`
Exploitante	`lea`, uid `5104`, commentaire `Lea Fabre – responsable suivi des tournees`, mot de passe de lab `Helios-Suivi-2026`
Compte de service	`helios-suivi` (compte système)
Répertoire du service · répertoire publié	`/srv/helios/suivi` · `/srv/helios/suivi/www`
Service HTTP	`helios-suivi.service`, commande imposée : `/usr/bin/python3 -m http.server 8080 --bind 0.0.0.0 --directory /srv/helios/suivi/www`
Publication	`/usr/local/sbin/helios-suivi-publish.sh`, `helios-suivi-publish.service` + `.timer`, tag journald `helios-suivi-publish`
Sauvegarde	`/usr/local/sbin/helios-suivi-backup.sh`, `helios-suivi-backup.service` + `.timer`, tag `helios-suivi-backup`, destination `/srv/helios/archives/suivi/`
Délégation sudo	`/etc/sudoers.d/92-helios-suivi`
Sondes Zabbix	`/etc/zabbix/zabbix_agent2.d/helios-suivi.conf`, clés `helios.suivi.http` et `helios.suivi.age`
Script d'audit	`/usr/local/sbin/helios-suivi-audit.sh`

3. Cahier des charges

Dix exigences. Chacune se vérifie par une commande ; c'est cette commande, et sa sortie, que vous collez dans `RAPPORT.md` (§5).

E1 — Comptes et groupe *(TP2, Mission 1)*

1. Le groupe `suivi` et le compte `lea` existent avec les identifiants imposés ; `suivi` est le groupe **primaire** de `lea`.
2. Le mot de passe de `lea` expire à 90 jours, ne peut pas être changé avant 1 jour, avec une alerte 14 jours avant ; le compte est désactivé 7 jours après expiration.
3. `lea` n'est membre ni de `wheel` ni de `ssh-admins`.
4. `helios-suivi` est un compte **système** (uid < 1000), sans shell de connexion. Il a pour répertoire personnel `/srv/helios/suivi` et ne crée aucun répertoire dans `/home`.

Vérifié par : `id`, `getent passwd`, `chage -l`.

E2 — Répertoire publié : setgid et ACL *(TP2, Missions 2 et 3)*

1. `/srv/helios/suivi/www` appartient à `root:suivi`, mode `2770`. Les « autres » n'y ont **aucun** droit.
2. `helios-suivi` peut lire **et écrire** dans `www` (c'est lui qui publie `etat.json`).

3. L'agent Zabbix (compte `zabbix`) doit pouvoir lire les métadonnées de `www/etat.json` (`stat`), **sans pouvoir lister** le contenu de `www`.
4. Tout fichier créé dans `www` par un membre de `suivi` appartient au groupe `suivi`, est modifiable par le groupe, lisible par `helios-suivi` et inaccessible aux autres, **sans aucune action après coup** et quel que soit l'`umask` de son auteur.
5. La page d'accueil `www/index.html` est déposée **par `lea` elle-même** (`su - lea`). Son contenu est libre (une phrase suffit).

Vérifié par : `ls -ld`, `getfacl`, un fichier créé par `lea`, `runuser -u <compte> -- ls/cat/stat`.

E3 — Script de publication *(TP1, Mission 3)*

`/usr/local/sbin/helios-suivi-publish.sh` lit la dernière ligne de `telemetry.csv` et produit `www/etat.json`, un objet JSON **valide** avec au moins les champs `genere` (horodatage ISO 8601 de la publication), `hote`, `derniere_mesure`, `charge_1min`, `mem_dispo_ko` et `mesures` (nombre de lignes).

1. Les cinq règles d'un script de production du TP1 : mode strict, verrou exclusif, nettoyage sur sortie (`trap`), journalisation vers journald sous le tag `helios-suivi-publish`, codes retour explicites.
2. Codes retour : `0` succès · `1` erreur interne (par exemple une ligne de télémétrie invalide) · `2` une autre instance tourne déjà · `3` prérequis manquant (télémétrie illisible, **ou** `www` non inscriptible pour le compte qui lance le script).
3. **Écriture atomique** : un lecteur ne doit jamais voir un `etat.json` à moitié écrit.
4. Le script fonctionne lancé en `helios-suivi`. Il fonctionne **aussi** lancé par son unit (E4), dans les contraintes de durcissement de celle-ci.

> 💡 Une unit en `ProtectSystem=strict` ne peut écrire que dans ses `ReadWritePaths`. Le verrou et le fichier temporaire doivent donc se trouver à un endroit où le script a le droit d'écrire, ou utiliser un mécanisme qui n'exige pas d'écrire.

Vérifié par : les trois codes `0`, `2` et `3` provoqués, `python3 -m json.tool`, `journalctl -t`.

E4 — Units systemd : portail durci et publication planifiée *(TP1, Missions 1 et 3)*

1. `helios-suivi.service` lance la commande imposée sous l'identité `helios-suivi`. Il est rattaché à `helios.target` (il démarre donc au boot avec la pile Hélios) et redémarre seul en cas de crash (testez-le avec `kill -9`).
2. Ressources : `MemoryMax=128M`, `CPUQuota=50%`.
3. Durcissement : `systemd-analyze security helios-suivi.service` donne une exposition **≤ 4.0**, et le portail répond toujours `200` sur `/` et sur `/etat.json`. Le serveur web n'a besoin d'écrire nulle part : son unit ne doit le lui permettre nulle part.
4. `helios-suivi-publish.service` (type `oneshot`, identité `helios-suivi`) ne peut écrire **que** dans `www`. Il est déclenché par `helios-suivi-publish.timer` **toutes les 2 minutes**.
5. Aucune unit livrée par un TP précédent n'est modifiée.

Vérifié par : `systemctl status` / `show` / `list-dependencies` / `list-timers`, `curl`, `systemd-analyze security`.

E5 — Délégation sudo *(TP2, Mission 4)*

Les membres du groupe `suivi` peuvent, et peuvent **seulement** :

1. redémarrer `helios-suivi.service` et afficher son état ;
2. forcer une publication (démarrer `helios-suivi-publish.service`) ;
3. lire les **100 dernières lignes** du journal de chacune de ces deux units.

Le fichier est validé par `visudo` **avant** installation et installé en `0440 root:root`. Aucune règle ne doit permettre d'ajouter des options aux commandes autorisées. Demandez-vous ce qu'un joker `*` derrière `journalctl` permettrait (pensez aux options `--vacuum-*`). Pas de `NOPASSWD`.

Vérifié par : `visudo -c`, `sudo -l -U lea`, `sudo -l -U lea <commande précise>` (autorisée/refusée), un essai réel en `lea`.

E6 — Pare-feu *(TP3, Mission 1)*

1. Le port `8080/tcp` est ouvert **uniquement** dans la zone `internal` (réseau métier), en runtime **et** en permanent. Il n'apparaît dans aucune autre zone, ni comme port ni dans une règle riche.
2. La zone par défaut reste `drop`.
3. Depuis `srv-edge` : `curl http://10.42.0.10:8080/etat.json` répond `200`, alors que `nc -zv -w2 192.168.56.10 8080` (réseau d'administration) échoue.

Vérifié par : `firewall-cmd --query-port`, `--list-all`, `nft list ruleset | grep 8080`, les deux tests depuis `srv-edge`.

E7 — Non-régression de la sécurité *(TP3, Missions 2 et 3)*

Après **toutes** vos modifications :

1. `sshd -T` donne toujours le port `2242`, `permitrootlogin no`, `passwordauthentication no` et `allowgroups ssh-admins`.
2. `lea` n'a **pas** d'accès SSH. Prouvez le refus et retrouvez la ligne de journal `sshd` correspondante.
3. `helios-watch-auth.timer` est actif et `journald` est toujours persistant.

E8 — Supervision *(guide : énoncé du TP4, §2.1 pour le dépôt et l'exclusion EPEL, §2.3 pour l'agent)*

1. **À faire en premier.** Vous installez `zabbix-agent2`, `zabbix-get` et `sysstat` (dépôt Zabbix 7.0 : suivez le guide du TP4, en particulier l'exclusion de `zabbix*` du dépôt EPEL). N'installez **pas** le serveur Zabbix ni MariaDB (hors bonus) ; sur `srv-core`, l'agent garde `Server=127.0.0.1`. Les services sont actifs et démarrent au boot. Le paquet crée le compte `zabbix`, dont E2 a besoin.
2. La collecte `sar` (`sysstat-collect.timer`) est active.
3. `helios-suivi.conf` définit deux `UserParameter` :
   - `helios.suivi.http` : code HTTP renvoyé par `http://127.0.0.1:8080/etat.json` (attendu `200`) ;
   - `helios.suivi.age` : âge en secondes de `etat.json` (attendu < 300).
4. `zabbix_get -s 127.0.0.1 -k helios.suivi.http` et `-k helios.suivi.age` renvoient ces valeurs **par l'agent en service**. Il s'exécute sous le compte `zabbix` : c'est E2.3 qui le rend possible.
5. *Bonus* (il suppose d'installer aussi le serveur Zabbix, TP4 Mission 1) : dans le frontend, les deux éléments sur l'hôte `Zabbix server` et deux déclencheurs (`helios.suivi.http` différent de 200 →

Haute ; `helios.suivi.age` > 300 → **Moyenne**), déclenchés pour de vrai en arrêtant le portail puis le timer.

E9 — Sauvegarde et restauration vérifiée *(TP2 3.4, TP1 Mission 3)*

0. `/srv/helios/archives` n'existe pas au départ : créez-le (propriétaire `helios:helios`, mode `0750`). Il n'y a pas de grappe RAID dans ce projet.
1. `/usr/local/sbin/helios-suivi-backup.sh` respecte les cinq règles du TP1 (tag `helios-suivi-backup`). Il produit `/srv/helios/archives/suivi/suivi-AAAAMMJJ-HHMMSS.tar.gz` **avec les ACL**, accompagnée d'un fichier d'empreinte `.sha256` vérifiable par `sha256sum -c`.
2. L'archive contient tout ce qu'il faut pour reconstruire le service : `/srv/helios/suivi`, les units du projet, le fichier sudoers, la configuration Zabbix et les deux scripts.
3. Seules les **7** archives les plus récentes sont conservées.
4. `helios-suivi-backup.timer` la déclenche chaque jour à 03 h 15 et rattrape une exécution manquée si le serveur était éteint.
5. **Restauration prouvée**, en deux temps, documentée dans `RAPPORT.md` : (a) extraction de la dernière archive dans un répertoire de test, et comparaison des ACL de `www` avec la production ; (b) incident simulé : supprimez `index.html`, puis restaurez **ce seul fichier** depuis l'archive, avec ses ACL, sans toucher au reste.

E10 — Script d'audit de conformité (BONUS)

> **Bonus** : à traiter une fois E1 à E9 terminées et `RAPPORT.md` rédigé. Non fait, il n'est pas pénalisant.

`/usr/local/sbin/helios-suivi-audit.sh` : c'est lui que le directeur technique lancera.

1. Lancé en root, il **ne modifie rien**. Il affiche une ligne par exigence E1 à E9, qui commence par `[OK]` ou `[KO]`, puis une dernière ligne `Bilan : n/9`.
2. Code retour `0` si et seulement si tout est conforme, différent de `0` sinon.
3. Il détecte au minimum : le portail arrêté ; `8080/tcp` retiré du pare-feu en runtime ; un `etat.json` vieux de plus de 5 minutes ; une délégation sudo qui autorise `systemctl restart sshd` ; une sauvegarde absente ou d'empreinte invalide.
4. Il ne doit pas afficher de faux `[OK]`. Un contrôle qui ne peut pas s'exécuter (commande absente, fichier illisible) est un `[KO]`.

Sa sortie sur votre serveur conforme est commitée dans `PROJET/preuves/audit.txt`.

---

4. Contraintes

- **Aucune régression** : les services, timers et règles des TP1 à TP3 restent actifs et inchangés. Ajoutez, ne remplacez pas.
- **Interdits**, quel que soit le motif : `chmod o+…` ou `777` pour contourner une ACL ; `NOPASSWD` ; arrêter ou désactiver `firewalld` ; ouvrir `8080` hors de `internal` ; `setenforce 0` ou `SELinux=permissive`. Si SELinux bloque quelque chose, diagnostiquez-le (`ausearch -m AVC -ts recent`) et documentez-le.
- **Tout est persistant** : rien ne doit dépendre d'une commande tapée après le boot. Si le temps le permet, `systemctl reboot` puis revérifiez E1 à E9 (avec votre audit si E10 est faite : il doit rester à

`9/9`).

- **Aucun secret dans le dépôt** : pas de clé privée, pas de mot de passe autre que ceux du lab.
- Les sorties collées dans `RAPPORT.md` sont **les vôtres**, copiées du terminal. Une sortie inventée ou retouchée est une fraude (§7).

5. Livrables

Dans votre dépôt Git, au plus tard **vendredi à 13 h 00** (le dernier commit poussé avant 13 h 00 fait foi) :

```
PROJET/
├── configs/      # les 5 units, les 3 scripts, 92-helios-suivi, helios-suivi.conf,
│                 # getfacl de www, firewall-cmd --zone=internal --list-all
├── preuves/      # audit.txt (sortie de votre audit, bonus E10), et toute sortie trop
longue pour le rapport
├── RAPPORT.md
└── RUNBOOK.md    # bonus
```

`RAPPORT.md` contient, dans cet ordre :

1. **Tableau de conformité** : une ligne par exigence (E1 à E9, et E10 si traitée) avec la commande de preuve et la partie utile de sa sortie, ou un renvoi vers `preuves/`.
2. **Trois justifications**, dix lignes au plus chacune : (a) votre durcissement de `helios-suivi.service`, le score obtenu, et la directive que vous avez dû retirer ou adapter parce qu'elle cassait le service (s'il y en a une) ; (b) pourquoi `--x` et pas `r-x` pour `zabbix` sur `www`, et ce que vous avez observé sans cette entrée ; (c) le risque précis d'un joker `*` dans une règle sudo `journalctl`, et comment votre règle l'évite.
3. **Test de restauration** (E9.5) : commandes, sorties, et le temps qu'il vous a fallu.
4. **Mesure sous charge** : pendant `for i in $(seq 2000); do curl -s -o /dev/null http://127.0.0.1:8080/etat.json; done`, relevez `sar -u` sur la fenêtre et `systemctl show -p CPUUsageNSec -p MemoryCurrent helios-suivi.service` avant/après. Le quota `CPUQuota=50%` a-t-il joué ? Répondez en deux lignes, chiffres à l'appui.
5. **Limites connues** : ce qui ne marche pas, ou pas complètement, et ce que vous feriez avec une journée de plus.

`RUNBOOK.md` (**bonus**), à l'usage de l'astreinte (une page au plus) : **quatre incidents minimum**.

1. Le portail ne répond plus.
2. `etat.json` est figé.
3. Le portail est injoignable depuis le réseau métier alors que le service tourne.
4. Il faut restaurer le portail après une suppression ou une corruption.

Pour chaque incident, quatre lignes au plus : **symptôme** (quelle alerte Zabbix ou quel constat), **diagnostic** (commandes), **réparation** (commandes), **vérification** (commande et résultat attendu). Écrivez-le comme une fiche que l'astreinte peut suivre sans vous avoir croisé.

6. Planning sur deux jours

Quand	Travail	Point de contrôle personnel
Jeudi, lancement	Lecture de la fiche, contrôle des prérequis, instantané `04-debut-`	Les 5 contrôles du tableau des prérequis passent

Quand	Travail	Point de contrôle personnel
(10 h 30)	`projet`, création du dépôt et de `PROJET/`, **installation de l'agent Zabbix (E8, point 1)**	
Jeudi	E1, E2, puis E3, E4	Un fichier créé par `lea` dans `www` est lisible par `helios-suivi`, pas par `nadia` ; `curl http://127.0.0.1:8080/etat.json` renvoie un JSON qui change toutes les 2 min
Jeudi, fin de journée	E5, E6, E7, premier commit et `git push`	Test depuis `srv-edge` fait, `sshd -T` inchangé
Vendredi, avant le rendu	E8, E9, fin du `RAPPORT.md` ; **bonus** si le temps le permet : E10, `RUNBOOK.md`	`zabbix_get` répond ; restauration de `index.html` réussie ; (bonus) votre audit sort `9/9`, puis `KO` quand vous arrêtez le portail
Vendredi 13 h 00	**Rendu** : dernier commit et `git push`	`git log -1` daté avant 13 h 00

Le cours du Jour 5 (3 h) a lieu le vendredi : visez d'avoir terminé E1 à E7 le jeudi. Poussez votre travail au fil de l'eau, pas seulement à la fin.

> ⚠️ **Après le rendu**, le formateur peut lancer son propre script de contrôle sur votre `srv-core` (environ une minute, en votre présence, **sans oral noté**). Ce script provoque deux pannes et les répare. Laissez `srv-core` allumé et ne le modifiez plus une fois le rendu poussé. Si votre VM ne démarre plus, vos preuves dans le dépôt (`preuves/`, tableau de conformité) servent de seule base.

Une exigence vous bloque plus de 20 minutes ? Notez-la dans « Limites connues », passez à la suivante, revenez-y à la fin. Une exigence documentée honnêtement comme incomplète vaut mieux qu'une exigence E10 jamais commencée.

---

7. Évaluation et règles de rendu

Cette fiche **ne contient pas de grille de notation** : le rendu est le dépôt Git (`PROJET/`), relu avec la sortie de votre script d'audit (bonus E10), si vous l'avez écrit. L'appréciation, et le poids du projet dans la note du module, sont fixés par l'équipe pédagogique de l'école. Les TP journaliers n'ont pas de barème propre : leurs critères de réussite servent à vérifier que vous êtes prêt pour ce projet.

Règles qui s'appliquent quel que soit le jugement :

- **Sécurité dégradée** (`firewalld` arrêté, `8080` ouvert hors de `internal`, SSH qui réaccepte `root` ou les mots de passe, SELinux désactivé) : le projet est considéré comme non conforme, même si le reste fonctionne. Livrer un service en ouvrant le serveur est l'erreur que ce module vise à rendre impossible.
- **Secret commité** (clé privée, mot de passe réel) : à retirer du dépôt, et signalé.
- **Sortie inventée ou retouchée** dans le dépôt : signalée à l'équipe pédagogique.
- **Commit après vendredi 13 h 00** : il n'est pas pris en compte.

Critères de réussite

- [ ] Les 5 contrôles de prérequis passent, l'instantané `04-debut-projet` existe.
- [ ] Un fichier créé par `lea` dans `www` est `…:suivi`, lisible par `helios-suivi`, illisible par `nadia`.
- [ ] `runuser -u zabbix -- ls /srv/helios/suivi/www` échoue, `runuser -u zabbix -- stat …/etat.json` réussit.
- [ ] `helios-suivi-publish.sh` renvoie `0`, `2` et `3` dans les situations prévues.
- [ ] `curl http://127.0.0.1:8080/etat.json` renvoie un JSON valide, de moins de 2 minutes.
- [ ] Après `kill -9` du portail, il est de nouveau `active` en moins de 10 s.
- [ ] `systemd-analyze security helios-suivi.service` ≤ 4.0.
- [ ] `sudo -l -U lea /usr/bin/systemctl restart sshd` ne liste rien.
- [ ] Depuis `srv-edge`, `10.42.0.10:8080` répond et `192.168.56.10:8080` est filtré.
- [ ] `sshd -T` est identique à la fin du TP3 sur les quatre directives de E7.
- [ ] `zabbix-agent2` et `sysstat-collect.timer` sont actifs ; `zabbix_get -s 127.0.0.1 -k helios.suivi.age` renvoie un nombre inférieur à 300.
- [ ] `index.html` supprimé puis restauré depuis l'archive retrouve ses ACL.
- [ ] *(Bonus E10)* Votre audit sort `Bilan : 9/9` et `rc 0`, puis un `[KO]` et `rc ≠ 0` quand vous arrêtez le portail.
- [ ] `PROJET/` contient `configs/`, `RAPPORT.md` (et, en bonus, `preuves/audit.txt`, `RUNBOOK.md`), poussés avant vendredi 13 h 00.